Gymnasium Oberwil

Maturarbeit

Bildsynthese von menschlichen Gesichtern mithilfe von Kuenstlicher Intelligenz

Eine Anwendung eines Autoencoders

Luis Wirth, 4a

Betreut von
Dr. Jonas Gloor

Abgegeben im September 2019

Inhaltsverzeichnis

1 Vorwort **3**

2 Einleitung **4**

3 Konventionen und Notation **5**

4 Machinelles Lernen - die Theorie **6**
4.1 Allgemeine Begriffe . . . 6
4.1.1 Daten . . . 6
4.1.2 Modelle . . . 6
4.2 Training . . . 8
4.2.1 Kostenfunktionen . . . 8
4.2.2 Gradientenverfahren . . . 8
4.2.3 Stochastisches Gradientenverfahren fuer Machine Learning . . . 10
4.2.4 Adam Optimizer . . . 10
4.3 Künstliche Neuronale Netze und Deep Learning . . . 11
4.3.1 Perzeptron . . . 11
4.3.2 Biologische Analogie . . . 12
4.3.3 Erweiterung der Kuenstlichen Neuronen . . . 12
4.3.4 Kuenstliche Neuronale Netze . . . 13
4.3.5 Vorwaertspropagierung . . . 14
4.3.6 Lineare Algebra . . . 14
4.3.7 Backpropagation . . . 14
4.3.8 Universal Approximation Theorem . . . 14
4.4 Autoencoder und Convolutional NN . . . 15
4.4.1 Bedeutung von Autoencodern . . . 15
4.4.2 Variational Autoencoder? . . . 15
4.4.3 Detangled Autoencoder? . . . 15

5 Entwicklung eines Autoencoders **16**
5.1 Tools . . . 16
5.1.1 Python . . . 16
5.1.2 Tensorflow . . . 16
5.1.3 Keras . . . 16
5.2 Idee . . . 16
5.3 Voraussetzungen . . . 16
5.3.1 Trainingsdaten . . . 16
5.4 Programmieren . . . 16

6 Diskussion? **17**
6.1 Bedeutung von Autoencodern . . . 17
6.1.1 Datenkompression . . . 17
6.1.2 Datengenerierung . . . 17
6.1.3 Eigenschaften erkennen . . . 17
6.2 Anwendungen . . . 17

7 Schlusswort **18**

testtest

1 Vorwort

2 Einleitung

Die Leitfrage lautet: **Wie können mithilfe von Machine Learning Bilder von künstlichen meschlichen Gesichtern generiert werden?**

3 Konventionen und Notation

NOTATION ERKLAEREN (Vektor, Matrix, etc.)

Falls ein (Zeilen- oder Spalten-) Vektor $\vec{v}$ als Argument in eine Funktion f gegeben wird, ist zu verstehen, dass die einzelnen Komponenten des Vektors die Parameter der Funktion sind. $f(\vec{v}) = f(v_1, \ldots, v_n)$

4 Machinelles Lernen - die Theorie

Der Themenbereich des **Machinellen Lernens** beschäftigt sich mit Algorithmen und mathematischen Modellen, welche von selber lernen, Probleme zu loesen. Hierzu werden keine expliziten Instruktionen fest einprogrammiert (engl.: hard coding), sondern das Modell wird trainiert und optimiert sich von selbst. Dabei wird das Modell mit Trainingsdaten befuettern, mithilfe dessen eine Korrelation zwischen den Inputdaten und den Outputdaten erlernt werden soll. Der Zusammenhang der Daten wird von den Modellen verallgemeinert und das Modell kann dann Vorhersagen fuer neue Daten machen.

4.1 Allgemeine Begriffe

4.1.1 Daten

Der **Trainingsdatensatz** beinhaltet gewisse **Attribute**, die Merkmalstypen der Inputdaten. Die **Features** sind dann die spezifischen Werte der Attribute eines Trainingssample. Die sogenannten **Labels** sind die erwarteten Outputs, welche der Algorithmus vorherzusagen hat. Diese Vorhersagen werden mit den Labels abgeglichen und so bewertet. Anhand der Bewertungen wird dann eine Optimierung des Modells vorgenommen. Unter korrekten Bedingungen (keine Ueberanpassung) findet kein Auswendiglernen der Trainingsdaten statt, sondern ein Generalisieren des Zusammenhangs, anhand von Mustern und Gesetzmassigkeiten.

Um eine Endgueltige Bewertung des Modells durchzufuehren, wird ein Testdatensatz, welcher nicht Teil des Trainingsdatensatz ist, genutzt um Vorhersagen zu machen. Dies garantiert, dass kein Auswendiglernen moeglich ist.

Die Inputs/Features werden in einem Vektor $\vec{x} = (x_1, x_2, \ldots, x_n)^T$ und die Outputs/Vohersagen in einem Vektor $\vec{y} = (y_1, y_2, \ldots, y_m)^T$ zusammengefasst. Das gleiche macht man mit den Labels, wobei diese mit $\vec{\hat{y}} = (\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_m)^T$ bezeichnet werden.

Am Beispiel eines Datensatz, welcher fuer Bilderkennung von Hunden und Katzen genutzt wird, werden diese Begriffe verdeutlicht. Die Inputdaten sind die Fotos von Hunden und Katzen. Deren Attribute sind die moeglichen RGB-Farbenwerte (drei Zahlen, welche von 0 bis 255 reichen) aller Pixel. Die Features sind die spezifischen RGB-Werte aller Pixel. Die Outputdaten sind die Labels, welche in diesem Fall angeben, ob ein Hund oder eine Katze auf dem Bild gezeigt wird. Dies wird meistens mit einer One-Hot-Kodierung bzw. einem One-Hot-Vektor gemacht. Jeder Labelart wird eine eigene Vektorkomponente zugewiesen. Falls das Label aktiv ist, ist die Komponent 1 ansonsten 0. Also Hund koennte als $\vec{l}_H = (1, 0)^T$ reprasentiert werden und Katze als $\vec{l}_K = (0, 1)^T$.

Abbildung 1: eine Veranschaulichung eines Machine Learning Modells

4.1.2 Modelle

Ein **Modell** ist eine mathematische Funktion $f\colon \mathbb{R}^n \to \mathbb{R}^m$ welche die Inputs auf die Outputs abbildet $\vec{y} = f(\vec{x})$. Im Zusammenhang mit Wertevorhersagen spricht man von einem Regressionsmodell. Das Verhalten dieses Modells wird bestimmt durch seine **Modellparameter** $\phi_1, \phi_2, \ldots$. Das Ziel ist es die Modellparameter so einzustellen, dass die Vorherrsagen $\vec{y}$ besser mit den Labels $\vec{\hat{y}}$ uebereinstimmen. Dies wird iterativ gemacht, indem immer wieder leichte Anpassungen an den Parametern vorgenommen werden, bis sie das gewunschte Resultat liefern.

Das wohl einfachste Regressionsmodell ist eine Regressionsgerade. Diese ist angemessen, falls ein einfacher linearen Zusammenhang zwischen den Features und den Labels besteht. $y = \phi_1 x + \phi_0$. Nun muessten nur noch ϕ_0 und ϕ_1 bestimmt werden.
Neben den gelernten Parametern, gibt es auch noch sogenannte **Hyperparameter**. Diese koennen nicht erlernt werden, sondern muessen manuell vor dem Training gewaehlt werden und koennen den Lernvorgang erheblich beeinflussen.

Fuer Machine Learning haben sich gewisse Modelle besonders guet etabliert, darunter waeren: Support Vector Machines, Evolutionäre Algorithmen, und Künstliche Neuronale Netze. Diese Arbeit wird sich vorwiegend mit Neuronalen Netzen auseinandersetzen.

Um den Modellbegriff noch etwas zu festigen wird ein Beispielmodell erlautert:
Das Modell soll Vorhersagen zum Wetter machen. Anhand von Luftfeuchtigkeit und Temperatur muss es eine Aussage treffen, ob es gerade regnet oder nicht.

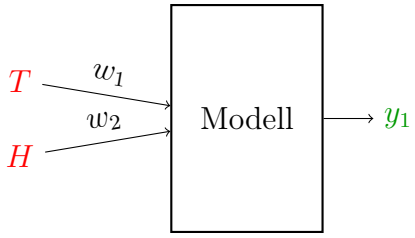

Abbildung 2: Darstellung des Modells

4.2 Training

4.2.1 Kostenfunktionen

Einsicht ist der erste Schritt zur Besserung. Das gilt auch beim Machine Learning, deshalb muss beim Training zuerst die Genauigkeit des Models bewertet werden. Dafuer gibt es sogennante **Kosten-, Verlust- oder Fehlerfunktionen**. Sie soll ein Mass fuer die Abweichung des Outputs $\vec{y}$ vom Label $\vec{\hat{y}}$ sein.

Die Fehler der einzlnen Outputs c_i werden aufsummiert um den Fehler C einer gesamten Vorhersage zu berechnen. Siehe Gleichung (1).
Der Fehler eines Datensatzen $\bar{C}$ ergibt sich aus dem arithmetischen Mittel der Vorhersagen-Fehler. Siehe Gleichung (2).

$$C(\vec{y}, \vec{\hat{y}}) = \sum_{i=1}^{m} c(y_i, \hat{y}_i) \qquad (1) \qquad\qquad \bar{C} = \frac{1}{p} \sum_{j=1}^{p} C\left(\vec{y}_j, \vec{\hat{y}}_j\right) \qquad (2)$$

Eine Kostenfunktion sollte folgende Eigenschaften aufweisen:

- c ist minimal, wenn $y = \hat{y}$
- c waechst mit $|\vec{y} - \vec{\hat{y}}|$
- C ist nach jedem y_n partiell differenzierbar

Eine beliebte Kostenfunktion ist die "Mittlere quadratische Abweichung" (engl.: mean squared error).
$C_{MSE} = \frac{1}{2n} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2 = \frac{1}{2n} (\vec{\hat{y}} - \vec{y})^2$ Sie erfuellt alle anforderungen, denn: Sie ist 0 falls $y = \hat{y}$. Sie ist proportional zu $(\hat{y} - y)^2$ und ihre Ableitung nach y lautet: $C' = \frac{1}{n}(\hat{y} - y)$

4.2.2 Gradientenverfahren

Um ein Modell nun zu trainieren, gilt es zu verstehen, das es sich um ein Optimierungsproblem handelt. Das Modell ist am besten, also macht die besten Vorraussagen, wenn die Fehlerfunktion am kleinsten ist. Deshalb muss diese Fehlerfunktion C minimiert werden. Hierbei muss die Funktion C nicht mehr in Abhangigkeit der Inputs betrachtet werden, sondern stattdessen in Abhangigkeit von den Modellparametern, $C(\phi_1, \phi_2, \ldots, \phi_n)$ denn diese sollen angepasst werden, um das Modell zu verbessern. Fuer diese Optimiertung wird das sogennante **Gradientenverfahren** (engl.: Gradient descent) verwendet. [1]

Allgemein koennen mithilfe des Gradientenverfahrens Funkionen $f(x_1, x_2, \ldots, x_n)$ vom Typ $\mathbb{R}^n \to \mathbb{R}$ (wie z.B. die Fehlerfunktion) minimiert werden. Dies beschieht, in dem ein Startpunkt (Ortsvektor) $\vec{p}_0$, dessen Komponenten den Input von f darstellt, gewaehlt wird. Nun werden iterativ neue Punkte $\vec{p_z}$ gesucht, welche immer naeher beim lokalen Minimun liegen, also Punkte die den Funktionswert $f(\vec{p}_z)$ immer kleiner werden lassen. Dies wird durchgeführt, bis der Punkt genug nahe beim lokalen Minimun ist.

Dafuer muss ein Vektor $\vec{b}_z$ bestimmt werden, welcher auf den Punkt $\vec{p}_z$ draufaddiert einen neuen Punkt $\vec{p}_{z+1}$ bildet, bei dem der Funktionswert $f(\vec{p}_{z+1})$ kleiner ist als der von $f(\vec{p}_z)$. Dies geschieht am effizientesten, wenn $\vec{b}_z$ in die Richtung der staerksten Funktionswertabnahme zeigt.

[1] Im Gymnasium wird beigebracht die lokalen Extrema (inkl. lokalen Minimas) zu bestimmen, indem die erste Ableitung f' gebildet wird und gleich null gesetzt wird. Dies ist hier nicht moeglich, da die Funktion $C'(\phi_1, \phi_2, \ldots, \phi_n)$ deutlich zu komplex ist, um die Nullstellen zu bestimmen. Deshalb wird das Gradientenverfahren verwendet.

Hierzu braucht man den sogennanten **Gradient** $\vec{\nabla}$, wofür man wiederum partielle Ableitungen braucht.

Partielle Ableitungen

Partielle Ableitungen sind eine Erweiterung der "normalen" Ableitungen auf multidimensionale Funktionen $f(x_1, \cdots, x_n)$. Man leitet dabei nur nach einem Parameter x_i ab und betrachtet die restlichen Argumente als Konstanten. Es gelten die gleichen Ableitungsregeln wie bei der nicht-partiellen Ableitung. Die partielle Ableitung einer Funktion $f(x_1, \cdots, x_n)$ bezueglich der Variabel x_i in einem Punkt $\vec{a} = (a_1, \cdots, a_n)^T$ ist analog zur "normalen" Ableitung folgendermassen definiert:

$$\frac{\partial f}{\partial x_i}(\vec{a}) := \lim_{h \to 0} \frac{f(a_1, \cdots, a_i + h, \cdots, a_n) - f(a_1, \cdots, a_i, \cdots, a_n)}{h}$$

Geometisch ist dies die Steigung der Tangente im Punkt $\vec{a}$ in Richtung der Achse des Parametern x_i, nach dem man ableitet.

Gradient

Der Gradient $\vec{\nabla}$ ist ein Differentialoperator, welcher ein Skalarfeld auf ein Vektorfeld (das sogennante Gradientenfeld) abbildet.
Dabei fasst man alle partiellen Ableitungen einer Funktion f in einem Vektor zusammen.

$$grad(f) = \vec{\nabla} f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix}$$

Geometisch ist der Gradient $\vec{\nabla} f(\vec{p})$ einer Funktion f in einem Punkt $\vec{p}$ dann der Vektor, welcher in die Richtung des steilsten Anstiegs von f zeigt. Seine Magnitude gibt die Staerke des Anstiegs an.

Da der Gradient in die Richtung des steilsten Anstiegs zeigt, sollte der Vektor $\vec{b}_z$ in die Richtung des negierten Gradient der Funktion f im Punkt $\vec{p}_z$ zeigen. Also kann jetzt das iterative Annahern an das lokalen Minimums so beschrieben werden:

$$\vec{p}_{z+1} = \vec{p}_z - \eta \cdot \vec{\nabla} f(\vec{p}_z) \tag{3}$$

Waehrend des Gradientenverfahren konvergiert der Punkt $\vec{p}_z$ zu einem belibigen *lokalen* Minimum, abhängig davon wie der Startpunkt $\vec{p}_0$ gewaehlt wurde. Da dieser meist zufaellig bestimmt wird, ist es eine Glückssache ein sehr tiefes Minimum zu finden.

Die sogennante **Lernrate** η aus Gleichung (3) ist ein Hyperparameter. Sie ist ein positiver Proportionalitatsfaktor, welcher die Schrittgroesse des Gradientenabstieg bestimmt. Sie muss je nach zu minimierender Funktion anderst gewaehlt werden. Dabei hilft nur ausporbieren. Falls η nicht gut gewahelt wurde, gibt es Probleme beim Training:

- Falls η zu klein ist, verlaueft das Trainings unnotig langsam und braucht sehr lange. Ausserdem kann es passieren, dass man bei einem hohen lokalen Minimum stecken bleibt.
- Falls η zu gross ist, passiert es, dass man ueber das lokale Minimum hinaus schiesst und nur darum herum springt.

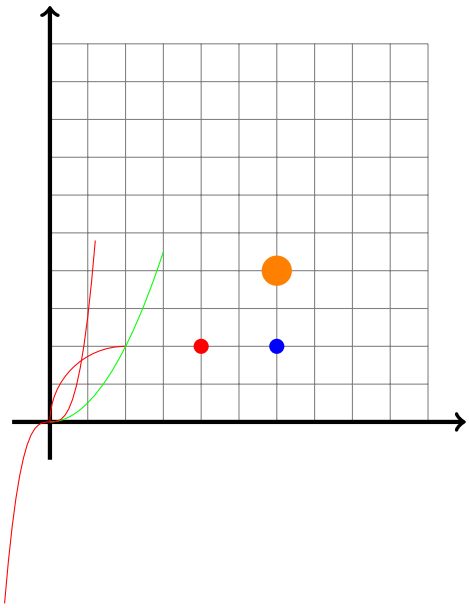

Abbildung 3: Visualiesierung verschiedener Lernraten

4.2.3 Stochastisches Gradientenverfahren fuer Machine Learning

Wie vorhin gesagt, wird fuer das Trainieren eines Modells das Gradientenverfahren benutzt. Konkret wird die Kostenfunktion $C(\phi_0, \ldots, \phi_n)$ minimieren, indem die Parameter angepasst werden, dadurch macht das Modell immer bessere Vorhersagen. Dafuer muesste man eigentlich den Gradieten fuer den gesamten Trainingsdatensatz bestimmen, um nur eine Iteration des Gradientenabstiegs durchzufuehren. Dies waere zwar ein exakter Prozess aber ein extrem langsamer zugleich. Bei grossen Datensaetzen wuerde es eine Ewigkeit dauern, bis das Modell nur annahernd guete Vorhersagen machen wuerde.

Aus diesem Grund verwendet man eine leicht abgeänderte Variante dieses Verfahren, naemlich das **Stochastische Gradientenverfahren** (engl.: Stochastic Gradient Descent). Dabei wird der "echte" Gradient des gesamten Datensatzes mit dem Gradienten einiger Trainingsbeispiel approximiert. Dabei wird der Trainingsdatensatz in sogennante **Mini-Batches** eingeteilt und der Gradient jeweils pro Mini-Batch berechnet. Dadurch finden deutlich mehr Iterationen statt in *einer* Durchkaemmung der Trainingsdaten, welche man **Epoche** nennt. Oft wird mehrere Epochen lang trainiert. Der Gradient eines genug grossen Mini-Batches ist zwar nicht ganz exakt, aber approximiert den Gradieten des gesamten Datensatzen genuegen gut. Sowohl die Mini-Batch Groesse, wie auch die Anzahl Epochen sind weitere Hyperparameter.

Die partiellen Ableitungen der gesamten Trainingsdaten wird mit dem arithmetischen Mittel der partiellen Ableitungen eines Mini-Batches der Groesse q approximiert. Siehe Gleichung (4).

$$\frac{\partial \bar{C}}{\partial \phi_n} \approx \frac{1}{q} \sum_{i=1}^{q} \frac{\partial C_i}{\partial \phi_n} \tag{4}$$

Eine Iteration des Stochastischen Graidentenverfahren wird analog zu Gleichung (3) folgendermassen durchgefuehrt:

$$\phi_{n,t+1} = \phi_{n,t} - \frac{\eta}{q} \sum_{i=1}^{q} \frac{\partial C_i}{\partial \phi_{n,t}} \tag{5}$$

4.2.4 Adam Optimizer

ZUSAMMENFASSUNG TRAININGSALGORITHMUS

4.3 Künstliche Neuronale Netze und Deep Learning

Das wohl beste Modell fuer die meisten Problemstellungen in Bereich Machinelles Lernen (Bilderkennung, Spracherkennung, etc.) ist das **Kuenstliche Neuronale Netz**, kurz KNN (eng.: Neural Network). Die Methoden welche benutzt werden um Neuronale Netze zu traniert fasst man unter dem Begriff **Deep Learning** zusammen.
Kuenstliche Neuronale Netze sind zum Teil biologisch inspiriert von Nervensystemen von Lebewesen. Sie sind aber lediglich eine Abstraktion der Informationverarbeitung und versuchen nicht eine moeglichst genaue biologische Abbildung darzustellen. Es gibt nicht nur eine Art von Neuralem Netz, sondern es exisitieren die verschiedensten Architekturen, welche je nach Problemstellung ausgewaehlt werden muessen. Diese Arbeit wird sich spaeter vorallem mit sogennanten Autoencodern auseinandersetzen.

4.3.1 Perzeptron

Um den Aufbau und die Funktion eines Kuenstlichen Neuronalen Netz besser zu verstehen, wird im folgenden ein Vorgaenger des KNN erklaert: das **Perzeptron**.

Das einlagige Perzeptron wurde erstmals 1958 von Frank Rosenblatt vorgestellt. Dieses besteht aus einem einzigen Kuenstlichen Neuron. Dieses Kuenstliche Neuron hat mehrere binaere Inputs und einen einzigen binaeren Output. Binaer bedeutet, dass der Wert nur entweder 0 (*aus*) oder 1 (*ein*) sein kann. Des weiteren besitzt es mehrere sogenannte **Gewichte** $w_1, \ldots, w_n \in \mathbb{R}$. Fuer jeden Input ein Gewicht. Diese sind reele Zahlen, welche das Verhalten des Perzeptron bestimmen. Die **Gewichtete Summe**, also die Summe aller Produkte der Inputs mit ihrem Gewicht, wird mit $\tilde{z}$ bezeichnet. Sie ist das gleiche wie das Skalarprodukt des Gewichtevektor mit dem Inputvektor. $\tilde{z} = \sum_{i=1}^{n} w_i x_i = \vec{w} \cdot \vec{x}$
Zusaetzlich besitzt das Perzeptron einen **Schwellenwert** $\tilde{b}$. Zusammen mit den Gewichten, bildet er die Modellparameter. Das Perzeptron verhaltet sich so, dass falls die Gewichtete Summe groesser als der Schwellwert ist, das Neuron feuert, d.h. der Output betraegt Output 1. Andernfalls ist er 0. Siehe erster Teil der Gleichung (6). Es ist gaengig die Ungleichung der Bedingung in die Nullstellenform zu bringen und $\tilde{b}$ durch die **Neigung** (engl.: Bias) $b = -\tilde{b}$ zu ersetzten. Somit lautet die Ungleichung: $\tilde{z} + b > 0$. Der Term $\tilde{z} + b$ wird mit z bezeichnet. Siehe Rest der Gleichung (6). Die Neigung gibt an wie stark das Neuron dazu neigt zu feuern. Ein hohes b laesst ein Neuron auch trotz einigen Nullen in den Inputs feuern, waehrend ein tiefes b es nicht feuern lassen wuerde.

$$p(\vec{x}) = \begin{cases} 1 & \text{falls } \tilde{z} > \tilde{b} \\ 0 & \text{ansonsten} \end{cases} = \begin{cases} 1 & \text{falls } \tilde{z} + b > 0 \\ 0 & \text{ansonsten} \end{cases} = \begin{cases} 1 & \text{falls } \vec{w} \cdot \vec{x} + b > 0 \\ 0 & \text{ansonsten} \end{cases} \tag{6}$$

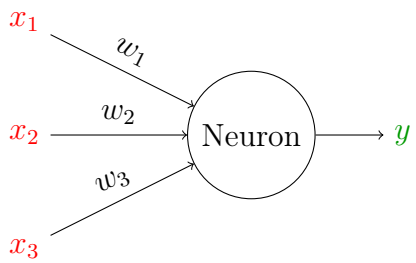

Abbildung 4: ein Perzeptron mit drei Inputs

Fuer das Trainieren des Perzeptron existieren spezielle Verfahren, welche hier aber nicht relevant sind. Das Gradientenverfahren kann naemlich nicht verwendet werden. Der Grund dafuer sollte spaeter einlauchtend sein.

Nun stellt sich die Frage, was ein Perzeptron erlernen kann und wofuer es genutzt werden kann. Das Perzeptron ist lediglich ein **Linearer Klassifikator** der Form $y = w_1x_1 + \ldots + w_nx_n$. Es kann die Features in zwei Klassen einordnen 0 oder 1. Ueberschreitet y den Schwellenwert, werden die Features der Klasse 1 zugeordnet, sonst der Klasse 0. Jedoch muessen diese Klassen linear separierbar sein. Das bedeutet dass, die Inputvektoren $\vec{x}$ durch eine Hyperebene trennbar sein muessen. Bei zwei Features (zwei Inputs) handelt es sich bei der Hyperebene um eine Gerade welche die Vektoren auftrennt. Siehe Abbildung (5). Fuer drei Featuers ist es eine Ebene.
Eine Funktionen, welche nicht linear separierbar ist (z.B XOR-Verknuepfungen), kann von einem Perzeptron nicht erlernt werden.

Abbildung 5: erfolgreiche lineare Separierung (links) und das Versagen bei XOR (rechts)

(**wiki:perzeptron**)

4.3.2 Biologische Analogie

————————- Gewicht entscheidet ob inhibitorisch oder exaktorisch. Schwellenwert: Das Addieren eines Schwellenwerts zur Netzeingabe verschiebt die gewichteten Eingaben. Die Bezeichnung bestimmt sich aus der Verwendung einer Schwellenwertfunktion als Aktivierungsfunktion, bei der das Neuron aktiviert wird, wenn der Schwellenwert überschritten ist. Die biologische Motivierung dabei ist das Schwellenpotential bei Nervenzellen. Mathematisch gesehen wird die Trennebene, die den Merkmalsraum auftrennt, durch einen Schwellenwert mit einer Translation verschoben.
Hebbsche Lernregel

4.3.3 Erweiterung der Kuenstlichen Neuronen

Ein Perzeptron ist, wie vorhin erklaert, nur in der Lage lineare Klassifikationen durchzufuehren. Um nun auch kompliziertere Probleme zu loesen, muss das Prinzip ausgebaut werden. Ausserdem brauchen wir ein Kuenstliches Neuron, welches sich besonders gut als Baustein fuer KNNs eignet.

Kuenstliche Neuronen im Allgemeinen Kuenstliche Neuronen sind immer so aufgebaut, dass sie einen oder mehrere Inputs haben und einen einzigen Output. Zu jedem Input x_i ist ein Gewicht w_{ji} assoziert. Zuerst wird die gewichtete Summe der Inputs $\tilde{z}$ gebildet. Die Neigung b wird ebenfalls draufaddiert, um z zu erhalten. Nun muss die sogennante Aktivierung a gebildet werden. Sie entspricht dem Output des Neurons. Die Aktivierung a, ist das Resultat der Aktivierungsfunktion φ angewendet auf z. $a = \varphi(z)$. Die verschiedenen Kuenstlichen Neuronen unterscheiden sich fast nur in ihrer Aktivierungsfunktion.

Perzeptronen als Kuenstliche Neuronen Zuerst nocheinmal ein Blick auf das Perzeptron in Betracht der Aktivierungsfunktion. Ein wesentlicher Unterschied des Perzeptron gegenueber sonstigen Kuenstlichen Neuronen, besteht darin, dass seine Inputs und Outputs nur binaere Werte annehmen koennen. Um dieses Verhalten des Perzeptron zu erhalten, muss eine Stufenfunktion als Aktivierungfunktion verwendet werden: die Heaviside-Funktion Θ. Sie hat einen einzigen Stufensprung bei $x = 0$ vom Wert 0 auf 1. Siehe Abb. (6).

$$\varphi^{\text{hlim}}(z) = \Theta(z) = \begin{cases} 1 & \text{falls } z \geq 0 \\ 0 & \text{falls } z < 0 \end{cases} \tag{7}$$

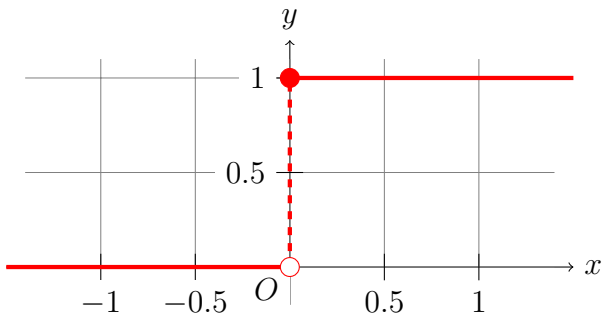

Abbildung 6: die Definition und der Plot der Heaviside-Funktion Θ

Stueckweise lineare Neuronen Der naechste Schritt nach dem binaeren Perzeptron, sind sogenannte stueckweise lineare Neuronen. Sie verwenden eine stueckweise lineare Aktivierungsfunktion. Diese bildet ein beschraenktes Intervall linear ab. Die Werte ausserhalb werden auf die konstanten Werte 0 oder 1 abbgebildet. Siehe Abb. (7).

Die Inputs koennen jetzt beliebige Reele Zahlen sein (vorzugsweise in der Naehe von 0 und 1). Ein KNN aus aussschliesslich linearen Neuronen bestehend, sind ziemlich nutzlos, da jedliche Komposition von linearen Funktion auch durch eine einzige lineare Funktion dargestellt werden koennte. Somit hat ein KNN gegenueber einem einzelnen Neuron keinen Mehrwert. Ausserdem koennen mit diesen Neuron auch nur lineare Probleme geloest werden.

$$\varphi^{\text{pwl}}(z) = \begin{cases} 1 & \text{falls } z > \frac{1}{2} \\ z + \frac{1}{2} & \text{falls } -\frac{1}{2} < z < \frac{1}{2} \\ 0 & \text{falls } z < \frac{1}{2} \end{cases} \tag{8}$$

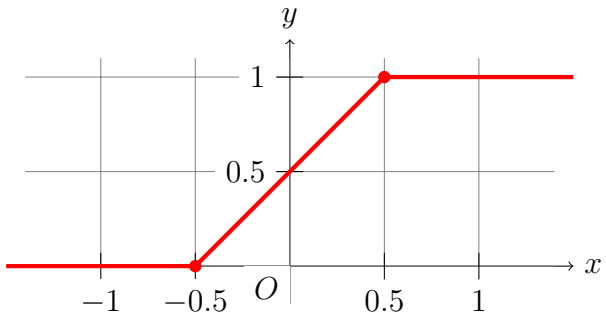

Abbildung 7: die Formel und der Plot einer stueckweisen linearen Funktion

Sigmoide Neuronen Die logische naechste Erweiterung sind nicht-lineare Neuronen. Ein sehr beliebter Kandidat dafuer sind Sigmoide Neuronen. Den Namen haben sie von ihrer Aktivierungsfunktion: der Sigmoidfunktion σ. Eine sehr relevant Eigenschaft von ihr ist, dass sie im Gegensatz zu den vorhin genannten Aktivierungsfunktion, ueberall differenzierbar ist. Dies ist eine Voraussetzung fuer das Gradientenverfahren und somit fuer das ganze Training. Zugleich ist sie monoton steigend und das sehr glatt. Auch andere Funktion erfuellen diese Bedingungnen. Die Sogmoidfunktion wird einfach bevorzusagt, da ihre Ableitung sehr simpel ist, siehe Abb. (8). Ihre nicht-linearitaet ermoeglicht ein erlernen von deutlich komplexeren Sachverhalten. Und wie spaeter erlautert, ist es moeglich mit der Kompostion von nicht-linearen Funktionen jede belibige Funktion zu approximieren!

Die Simgmoid-Funktion besitzt eine einzige Wendestelle bei $x = 0$ und hat zwei Asymptote, eine fuer $x \to -\infty$ mit $y = 0$ und eine zweite fuer $x \to \infty$ mit $y = 1$. Siehe Abb. (8).

4.3.4 Kuenstliche Neuronale Netze

Nun sollen diese Neuronen miteinander verbunden werden und ein Netz bilden, aehnlich wie ein Nervensystem. Diese Neuronen sind in verschieden Schichten (engl.: Layers) arangiert. Die

$$\varphi^{\text{sig}}(z) = \sigma(z) = \frac{1}{1+e^{-z}} \tag{9}$$

$$\sigma'(z) = \sigma(z)(1-\sigma(z)) \tag{10}$$

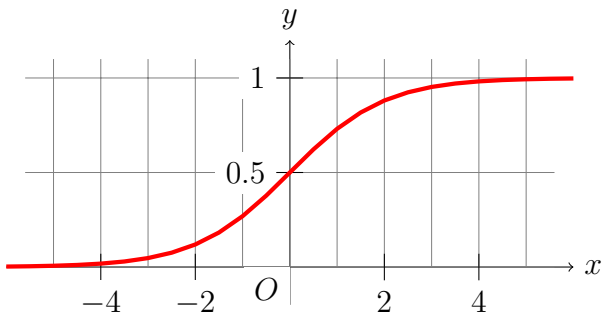

Abbildung 8: die Formel und der Plot der Sigmoid-Funktion σ

erste ist die **Inputschicht**. Sie behinhalten die Inputneuronen mit jeweils einem Inputwert. Dies sind eigentlich keine richtigen Neuronen sondern eher Platzhalter fuer die Inputs. Als letztes kommt die **Outputschicht** mit den Outputneuronen, welche jeweils einen Outputwert besitzen. Dazwischen liegen die **Zwischenschichten** (engl.: Hiddenlayer). Von ihnen kann es beliebig viele geben und in ihnen bliebig viele Neuronen. Jedes Neuron aus einer Schicht ist mit jedem Neuron aus der naechsten Schicht ueber Verbindungen gekoppelt. Alle Verbindungen besitzten ein Gewicht analog zum Perzeptron. Die Aktivierung, also der Output, eines Neurons wandert entlang den jeweiligen Verbindung zu allen Neuronen der naechsten Schicht und dient als deren Input. Die soeben beschriebe Art von KNN nennt man **Feedforward-Netz**, da alle Werte nur nach vorne propagiert werden.

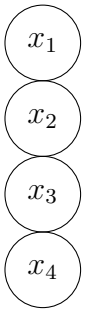

Abbildung 9: eine Darstellung der Schichtung eines KNN

4.3.5 Vorwaertspropagierung

Nun muessen einige Konventionen zur Beschriftung der Teil eines KNN getroffen werden.

- a_n^l ist die Aktivierung des n-ten Neurons in der l-ten Schicht.
- $w_{n,m}^l$ ist das Gewicht der Verbindung zweischen dem n-ten Neuron in der l-ten Schicht und dem m-ten Neuron in der $l+1$-ten Schicht.

4.3.6 Lineare Algebra

4.3.7 Backpropagation

4.3.8 Universal Approximation Theorem

Neural Network kann alles erlernen

4.4 Autoencoder und Convolutional NN

4.4.1 Bedeutung von Autoencodern

4.4.2 Variational Autoencoder?

4.4.3 Detangled Autoencoder?

5 Entwicklung eines Autoencoders

5.1 Tools

5.1.1 Python

5.1.2 Tensorflow

5.1.3 Keras

5.2 Idee

5.3 Voraussetzungen

5.3.1 Trainingsdaten

MegaFace Dataset

5.4 Programmieren

6 Diskussion?

6.1 Bedeutung von Autoencodern

6.1.1 Datenkompression

Hauptkomponentenanalyse und Dimensionalitatsreduktion

6.1.2 Datengenerierung

6.1.3 Eigenschaften erkennen

6.2 Anwendungen

7 Schlusswort

Abbildungsverzeichnis

1 eine Veranschaulichung eines Machine Learning Modells 6
2 Darstellung des Modells 7
3 Visualiesierung verschiedener Lernraten 10
4 ein Perzeptron mit drei Inputs 11
5 erfolgreiche lineare Separierung (links) und das Versagen bei XOR (rechts) . . . 12
6 die Definition und der Plot der Heaviside-Funktion Θ 13
7 die Formel und der Plot einer stueckweisen linearen Funktion 13
8 die Formel und der Plot der Sigmoid-Funktion σ 14
9 eine Darstellung der Schichtung eines KNN 14

Tabellenverzeichnis

Selbstständigkeitserklärung

Ich erkläre hiermit, dass ich diese Arbeit selbständig durchgeführt und keine anderen als die angegebene Quellen, Hilfsmittel und Hilfspersonen beigezogen habe. Alle Textstellen in der Arbeit, die wörtlich oder sinngemäss aus Quellen entnommen wurden, habe ich als solche gekennzeichnet.

Luis Wirth